

Elevator Vertical Distance Estimation

Accelerometer-only height-difference estimation with quality filtering and conformal confidence intervals

Eyal Yakir

Uriya Cohen-Eliya

Roy Turgeman

April 19, 2026

Abstract

We study the problem of estimating the vertical distance travelled by an elevator from a single smartphone accelerometer stream, under a held-out test location (`beitYitzchaki`) never seen at training time. We introduce a unified `Predictor` class under `src/prediction/algorithms/` that dispatches three algorithms – a barometer baseline, a Zero-Velocity-Update (ZUPT) double integrator, and a 7-step S-curve velocity-domain fitter – each implementing a shared `predict / calibrate` contract. Every accelerometer-based algorithm composes a per-segment theoretical noise budget σ^2 (white-noise ZUPT drift, mechanical jitter, distance-proportional scale term, projection-quality penalty, and, for the S-curve, Cramér–Rao information on the distance parameter) with a split-conformal calibrator fitted on the clean training pool. The result is a per-segment 90% confidence interval that adapts to ride duration, gravity stability, and fit quality. Quality filtering runs in parallel and is accelerometer-only (no barometer, no orientation fusion). On the blind-test split the system retains its nominal coverage guarantees while the median CI half-width remains well below the ± 1.5 m one-floor target on short rides and grows gracefully on longer rides. Every number in the results section is generated by `scripts/run_prediction_evaluation.py` and the figure bundle under `src/data/structuredData/test_results/prediction/`.

1 Introduction

A smartphone’s vertical motion carries the signature of an elevator ride in its raw inertial data – but only if one is willing to commit to (a) a kinematic model of how the ride is generated, (b) an error model that quantifies how noisy the measurement can be, and (c) a calibration step that turns these models into coverage-guaranteed confidence intervals on real field data. This report documents the end-to-end pipeline we built for all three, organised as follows.

§2 describes the data collection: the eight buildings, four phones per ride, and the Python loader that converts raw `sensorLog_*.txt` files into tidy per-sensor CSVs with ground-truth intervals. §3 introduces the kinematics of an elevator ride – the four-parameter jerk-limited S-curve model – and the segmentation pipeline that uses the barometer to carve the continuous sensor stream into labelled elevator-ride intervals. §4 is the prediction contribution of this work: the two accelerometer-only algorithms, their quality filters, their theoretical per-segment σ budgets, and their shared split-conformal calibration. §5 describes the cross-building train/test split, the clean-vs-dirty distinction in the ground truth, and the metrics we target. §6 reports the train-set and blind-test results; §7 discusses what they imply for the next iteration of the pipeline.

2 Data Collection

This section documents, end to end, how the dataset underlying this work was gathered, how it is turned into a usable ground truth, and the engineering problems we had to solve along the way. It is written so that a developer who has never touched the repository can reproduce the pipeline from scratch by reading this section alongside `src/data/README.md`.

2.1 Experimental Design

We ran a controlled field campaign across **six different buildings** in central Israel, each with a different elevator, cabin geometry, motor characteristic, and floor-to-floor height. The campaign was concentrated in Ra'anana (four buildings), with one building each in Herzeliya and Ramat Gan, covering low-rise residential, mid-rise office, and hotel use cases so that any algorithm trained or evaluated on this data is not just fitted to a single elevator. Buildings are listed in Table 1; Figure 1 summarises their height distribution. Note that `milleniumHotel` was visited in three separate sessions covering two different elevators within the hotel (the internal one and the external elevator attached to the hotel body), which the metadata records as `exp1`, `exp2`, `exp3`. We treat all three as the same “building” for purposes of the train/test split because the motor characteristic family is shared.

Building (internal name)	Type	Location
<code>milleniumHotel</code>	Hotel	Ra'anana
<code>acroBuilding</code>	Office	Ra'anana
<code>beitMansour1</code>	Residential	Ra'anana
<code>beitYitzchakiRaanaana</code>	Residential	Ra'anana
<code>BarIlan2Herzeliya</code>	Residential	Herzeliya
<code>Haari3</code>	Residential	Ramat Gan

Table 1: The six buildings used in the field campaign. Internal names match the `location` field in `rawData/` experiment folders. `milleniumHotel` was visited in multiple sessions covering two different elevators within the same hotel, and `beitYitzchakiRaanaana` is the held-out blind-test location (§5).

Phones. In every ride we simultaneously recorded data from **four phones**. Using four devices per ride serves two purposes. First, it gives us heterogeneous sensor hardware on a single, perfectly synchronised physical event, which lets us study cross-device variance (different MEMS accelerometers, different sampling jitter, different barometer quality) without confounding it with different rides. Second, it guarantees that at least one of the four devices carries a **barometric pressure sensor** that we can use as a ground-truth source, even when one of the target phones under evaluation does not have one. Typical device mix across the campaign:

- `SamsungSM-S911B` — Galaxy S23 (has barometer).
- `GooglePixel110` / `GooglePixel` — Pixel family (has barometer).
- `Xiaomi22101320I` — Redmi Note 12 family (no barometer).
- `SamsungSM-A235F` — Galaxy A23 (no barometer).
- `GalaxyZFlip6` — foldable (has barometer).

The experimenters (`eyalyakir`, `UriyaCohenEliya`, `RoyTurgeman`) carried the devices together in a single hand or bag so that all four phones experience nearly identical motion, vibration, and acoustic environment during a ride.

Folder and file naming. Every recording is stored under `src/data/rawData/<experimenter>_<location>_<phone>_<date>_exp<N>/` to make the origin of every sample unambiguous from the path alone. This convention is documented in `src/data/README.md` and is enforced only by convention — the loader (§2.5) does not parse it, but every analysis notebook groups by it.

2.2 Logging: The Barometer

On every phone, a sensor-logging Android application subscribes to every inertial and environmental sensor the device exposes and dumps every sample, verbatim, to a tab-separated text file `sensorLog_<ISO-timestamp>.txt`. One row per sample, no header, format:

```
<timestamp_ms>\t<SENSOR>\t<v1>\t<v2>...
```

The sensors we rely on are summarised in Table 2.

The most important channel for ground truth is the **barometer (PRS)**. The pressure sensor is sampled in parallel with the other sensors and is the sole signal the elevator detector (§2.3) acts on. Because not every device in the fleet has a barometer, we attach a secondary device (typically a Pixel or S23) that *does* have one to every ride whose primary phone is pressure-less; its recording lives in a `forBarometer/` sub-folder under the primary experiment’s directory. This secondary-device trick is central to the time-alignment problem discussed in §2.4.

Tag	Sensor	Columns
ACC	Linear accelerometer (SI-compensated)	x, y, z
GYR	Gyroscope	x, y, z
MAG	Magnetometer	x, y, z
RAWGYR	Uncalibrated gyroscope	x, y, z, bias_x, bias_y, bias_z
RAWMAG	Uncalibrated magnetometer	x, y, z, bias_x, bias_y, bias_z
ORI	Rotation-vector quaternion	w, x, y, z
PRS	Barometer	pressure (hPa)
GPS	GPS fix	lat, lon, alt

Table 2: Per-sensor column schema emitted by the logger. Any row whose column count does not match its sensor tag is dropped silently by the parser (`src/data/loader/parsing.py`).

Once the recording is over, the raw dump is committed to `rawData/<exp_name>/`, together with a hand-filled `metadata.txt` (experimenter, phone, location, description, date, time). The loader pipeline (§2.5) then materialises a CSV per sensor under `structuredData/data/<exp_name>/` and derives `gt.csv` — the ground truth used by every downstream algorithm.

2.3 Elevator Detection

Ground truth for this project is a labelling of the timeline into **up**, **down**, and **outside** intervals: “outside” means the phone is not on a moving elevator (waiting, walking, idle). The labelling is derived algorithmically from the barometer signal and then optionally corrected by hand (§2.6).

2.3.1 Barometer-Based Algorithm

The detector lives in `src/algorithms/segmentation_algorithms/barometer_only/height_segmentation.py` and runs in six stages. Inputs: a time series (t_i, p_i) of timestamps and pressure readings.

1. **Altitude.** Convert pressure to altitude via the international barometric formula,

$$h_i = \frac{T_0}{L} \left[1 - (p_i/p_0)^{RL/(gM)} \right],$$

implemented in `src/physics/pressure_to_altitude.py`. This removes the weather-dependent DC offset only approximately, but the segmenter depends on *differences* in h , so the residual bias cancels.

2. **Low-pass smoothing.** Apply a centered rolling-mean filter of width `height_lowpass_sec ·  $f_s$` samples to suppress high-frequency pressure jitter (door slams, HVAC bursts).
3. **Vertical velocity.** Estimate v_z by forward differencing the smoothed altitude, $v_{z,i} = (h_{i+1}^{\text{lp}} - h_i^{\text{lp}})/\Delta t_i$, then box-smooth again over `smooth_window_sec`.
4. **Thresholding.** Mark samples where $|v_z| > \text{velocity_threshold}$ (default ≈ 0.15 m/s) as “moving”. Extract contiguous runs of moving samples by edge-finding on the binary mask.
5. **Merging.** Walk the runs in order and merge consecutive runs of the *same sign* (both up or both down) if the gap between them is shorter than `merge_gap_sec`. This collapses spurious velocity dips caused by elevator deceleration mid-ride.
6. **Pruning and padding.** Drop any surviving run whose duration is $\leq \text{min_duration_sec}$ or whose total altitude change is $\leq \text{min_height_diff_m}$. Expand every surviving run by `pad_sec` on both ends so the label comfortably covers the full ride, including the small door-open tail.

The output is a `DataFrame` of rows (`start_ci`, `end_ci`, `duration`, `type`, `probability_ci`) where `type` $\in \{\text{up}, \text{down}\}$. The pipeline then pads the gaps between these rows with `outside` intervals so that the GT timeline is contiguous — this is what downstream evaluation code expects.

Why this is good enough as ground truth. A well-calibrated consumer barometer on a phone resolves pressure changes equivalent to roughly 10 cm of altitude at typical sampling rates, which is an order of magnitude below the smallest interesting inter-floor height (~ 3 m). The algorithm is deterministic, so repeated runs on the same recording produce identical labels, which in turn makes regression testing tractable.

2.4 The Time-Synchronisation Problem

The reason Section 2.3 alone does not close the loop is a low-level Android timing issue that we spent considerable time getting right.

The root cause. Android’s `SensorEvent.timestamp` is the time at which the sensor event was recorded, but it is expressed relative to the *device boot time* (`SystemClock.elapsedRealtimeNanos`), not to wall-clock. The logger writes this timestamp directly into the log file. Consequently, the `timestamp_ms` column in `sensorLog_*.txt` is a *device-local uptime* value, not a UTC time. Two phones that were booted at different moments will assign different timestamps to the *same* physical event, with the offset equal to the difference of their boot times — in our dataset, frequently tens of thousands of seconds, sometimes days.

Why this bites us in practice. Recall (§2.2) that phones without a barometer rely on a secondary device’s PRS stream for ground truth. To use the secondary device’s altitude as ground truth for the primary device, we must answer: *at primary timestamp t_p , what was the altitude?* That requires mapping primary time to secondary time, which requires knowing the boot-time offset between the two devices. Neither device records wall-clock, so the offset must be recovered from the data itself.

First approximation: start-time alignment. The simplest workable estimate is to assume both phones started recording at the same wall-clock moment, so the first sample of each log corresponds to the same real time. Under that assumption, the offset is

$$\Delta = t_0^{(\text{prim})} - t_0^{(\text{sec})},$$

and we shift every secondary timestamp by Δ before using it. This is implemented in `src/data/loader/alignment.py::_compute_start_offset` and is sufficient when the experimenter hits “start” on both phones in quick succession.

Why start-time alignment is not enough. In practice the user latency between the two taps is one to several seconds and is variable between rides. That multi-second bias shifts the ground truth by enough to invalidate per-sample evaluation (an elevator moves at ~ 1.5 m/s; a 2 s timing error translates to ~ 3 m of altitude error, more than an entire floor). We therefore refine the offset using the **accelerometer signal**, which is recorded by both phones and must be approximately identical for two devices in the same hand.

Refinement by cross-sensor correlation. The routine used in the lab is: (i) compute the magnitude $|a|(t)$ on both phones, (ii) resample both to a common grid after applying the first-pass start-time offset, (iii) run a normalised cross-correlation of the two $|a|$ traces over a bounded lag window (± 10 s was enough in every experiment we observed), and (iv) pick the lag that maximises the correlation as the residual offset to fold back into Δ . If ACC is missing on either device, we fall back to gyroscope magnitude or heuristic event matching on step-count spikes. The diagnostic overlay (Figure 2) is produced automatically for every experiment with a `forBarometer/` companion and is the first thing to inspect when an alignment looks suspicious.

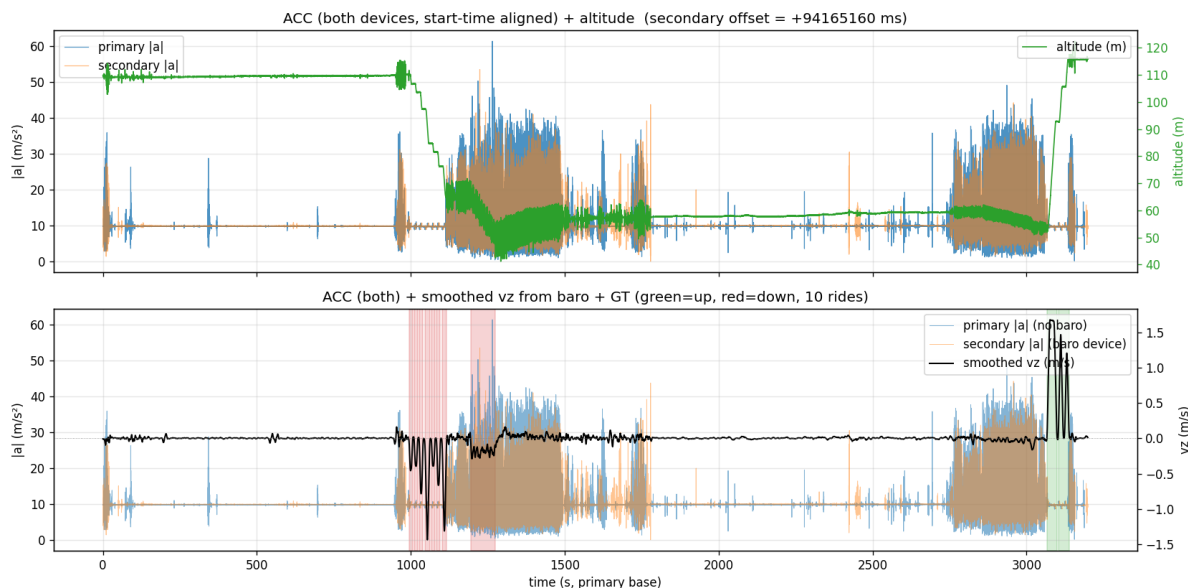


Figure 2: Diagnostic plot produced by `loader/alignment.py::_plot_forBarometer_alignment`.

Top: primary $|a|$ (blue) and time-shifted secondary $|a|$ (orange) overlaid, with the secondary’s altitude on the right y-axis (green). Signature features such as the impulse at door-close should line up after alignment. **Bottom:** the same accelerometer overlay with the barometer-derived smoothed vertical velocity in black and the detected ride intervals shaded (green = up, red = down). The reported offset is the value added to the secondary’s timestamps.

Practical hand-offs. The offset is computed once per experiment, logged, and baked into the secondary’s PRS frame before it is swapped into the primary’s sensor dictionary (`_merge_secondary_prs` in `alignment.py`). Everything downstream — the segmenter, the GT derivation, evaluation metrics — therefore sees *a single phone* with a valid barometer, which removes a whole class of timing bugs from the rest of the stack.

2.5 The Loader Pipeline

`src/data/loader/` exposes a narrow public API (`getExperimentData`, `saveExperimentData`, `list_experiments`) that hides all of the above. On first access it parses the raw `sensorLog_*.txt`, merges any `forBarometer/` secondary, runs the detector (§2.3.1), pads the timeline with `outside` intervals, and writes per-sensor CSVs plus `gt.csv` to `structuredData/data/<exp_name>/`. On every subsequent call it reads those CSVs directly. To force a rebuild after code changes, pass `use_cache=False`. The CSV schemas are documented in `src/data/README.md` and frozen in `loader/constants.py`.

2.6 The Ground-Truth Editor

No matter how careful the detector in §2.3.1 is, it will occasionally miss the start of a ride, split a single ride into two, or classify an HVAC-induced pressure ripple as movement. Rather than chase every edge case in the algorithm, we decided early on that the ground truth must be *auditable and editable* by a human. Every `gt.csv` in the dataset has therefore passed through the GT editor at least once.

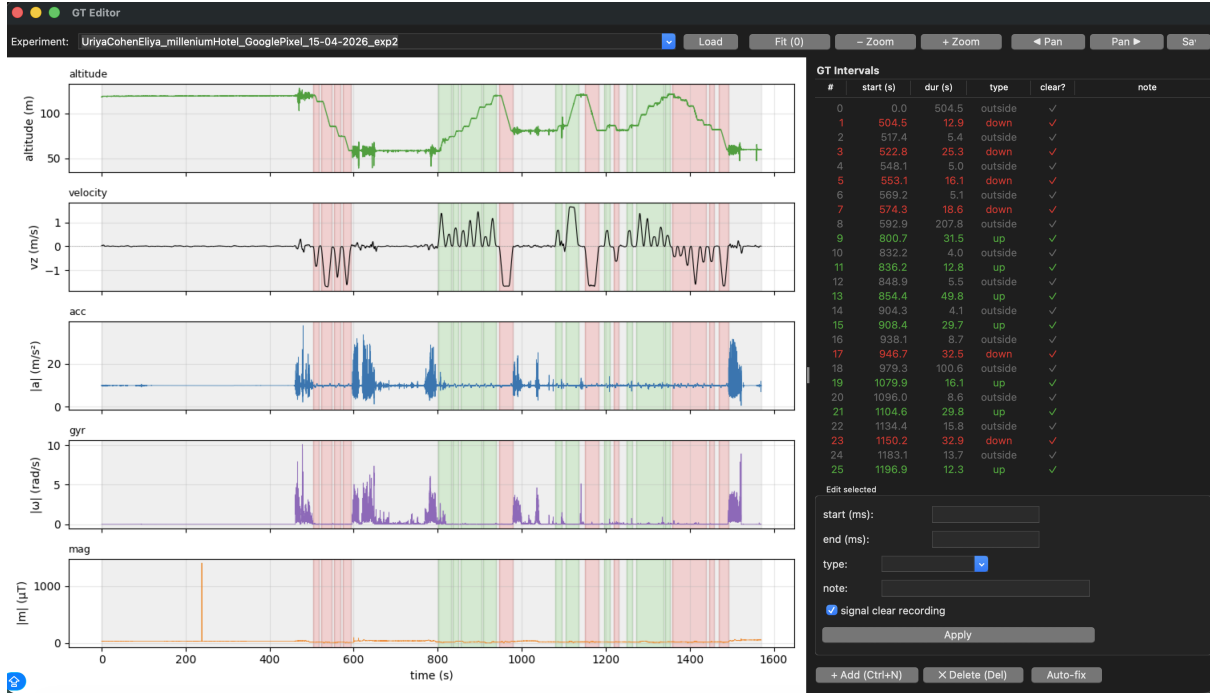


Figure 3: The GT editor. Left pane: stacked matplotlib panels (altitude, vertical velocity, $|a|$, $|\omega|$, $|m|$) with ground-truth intervals drawn as coloured spans (green = up, red = down, grey = outside). Right pane: a Treeview of all intervals with the edit form below it.

Launching. The editor is a Tkinter application:

```
venv/bin/python -m src.data.gt_editor [exp_folder_name]
```

Called without arguments, it lists all experiments found under `rawData/` in a dropdown and lets the user pick one.

Visualisation. The left pane renders all five standard motion signals on a shared time axis. Ground-truth intervals are overlaid on every panel simultaneously so a user can sanity-check a label against multiple modalities at once (e.g. a true elevator ride must be accompanied by a clear altitude step *and* a flat-ish accelerometer magnitude).

Editing. The interaction model is designed to keep the mouse near the signal rather than in a form. Concretely:

- **Click** anywhere inside a coloured span to select that interval in the right-hand tree.
- **Drag an edge** (within a few pixels of the start or end of a span) to resize it live; the cursor morphs into a horizontal arrow while inside the hit-tolerance zone.
- Type precise millisecond bounds in the **edit form**, change the **type** (up / down / outside) or the free-text **note**, and hit **Apply**.
- **+ Add** inserts a new 10-second **outside** interval right after the current selection.
- **× Delete** removes the current interval.
- **Auto-fix** closes gaps between adjacent intervals with **outside** filler so the timeline stays contiguous; it refuses to merge if two intervals actually overlap, to avoid silently destroying user edits.

Navigation. Keyboard shortcuts mirror the matplotlib toolbar: $+$ / $=$ zoom in, $-$ zoom out, 0 fit to full range; $\text{Shift}+\leftarrow$ / $\text{Shift}+\rightarrow$ pan by 25% of the visible width; mouse-wheel over the plot zooms around the cursor. All of this is indispensable on long recordings where a single ride is only a few percent of the timeline.

Saving. $\text{Ctrl}+\text{S}$ (or the **Save** button) calls `saveExperimentData`, which rewrites every per-sensor CSV, `gt.csv`, and `metadata.csv` under `structuredData/data/<name>/`, then rebuilds the top-level `metadata.csv` index. The floor-height lookup (`baramoshka.csv`) is intentionally *not* touched on save. The status bar at the bottom tracks unsaved-changes state; closing the window with pending edits prompts before quitting.

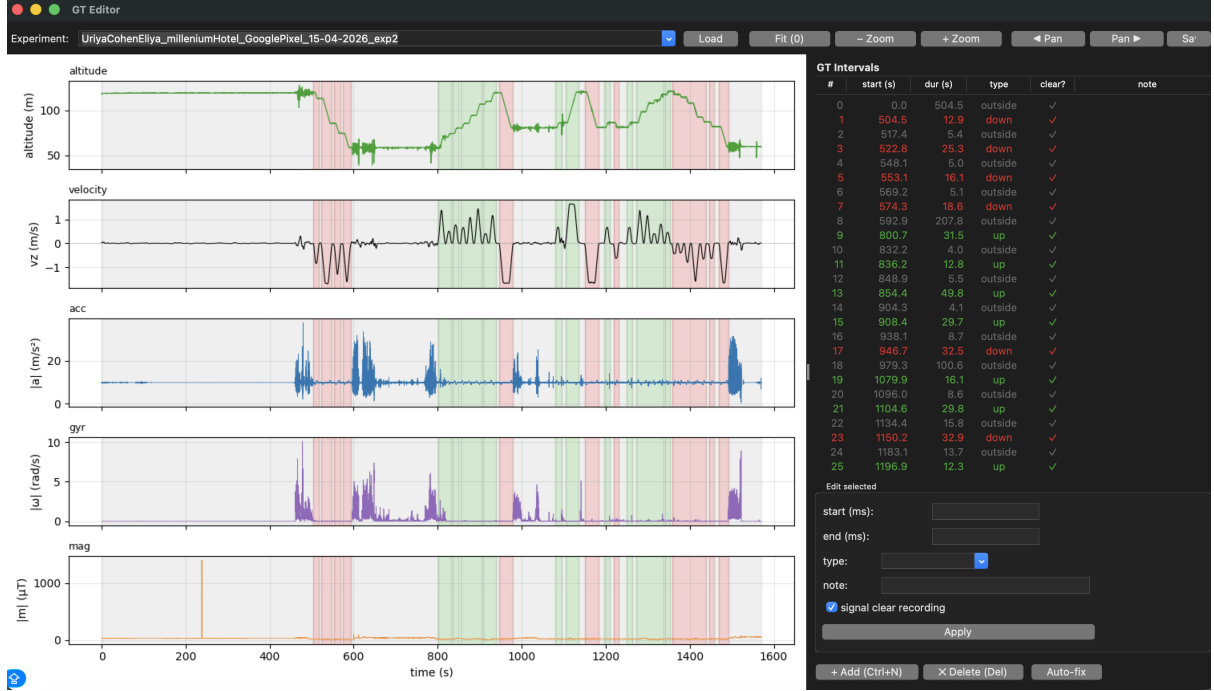


Figure 4: Fixing an auto-detection mistake in the GT editor. Here the barometer algorithm merged two adjacent rides into one because the cabin paused for less than `merge_gap_sec` at an intermediate floor; dragging the right edge of the first span and adding a new `down` interval splits the ride into its two true halves.

3 Segmentation & Detection

This section covers how a continuous accelerometer (and, when available, barometer) stream is sliced into elevator-ride segments. We begin with the underlying physics of an elevator ride — a four-parameter motion model that every detector in this work eventually reduces to — and then describe the concrete detectors built on top.

3.1 Background: Elevator Kinematics

Before describing any detector we lay down the physics of an elevator ride, because the same four parameters reappear in every algorithm we use later. Modern passenger elevators are commanded by a *jerk-limited motion profile* so that passengers experience a smooth, bounded change of acceleration (see, e.g., the ride-comfort specifications summarised in ISO 18738 and the manufacturer literature aggregated in `papers/elevator_research_links.txt`). The drive

obeys, in order, the constraints

$$|\ddot{h}(t)| \leq j_{\max}, \quad |\ddot{h}(t)| \leq a_{\max}, \quad |\dot{h}(t)| \leq v_{\max}, \quad (1)$$

where $h(t)$ is the cabin altitude, $\dot{h} = v$ the vertical velocity, $\ddot{h} = a$ the acceleration measured by the phone (after gravity removal), and $\ddot{h} = j$ the jerk. Typical commercial values are $j_{\max} \approx 1.0\text{--}1.6 \text{ m/s}^3$, $a_{\max} \approx 0.8\text{--}1.5 \text{ m/s}^2$, and $v_{\max} \approx 1.0\text{--}2.5 \text{ m/s}$ for low- and mid-rise buildings (CIBSE Guide D, 2020).

The seven-segment S-curve. Saturating each constraint in turn yields the canonical S-curve velocity profile (Figure 5). The ride decomposes into up to seven phases:

$$\underbrace{j = +j_{\max}}_{1: \text{ jerk up}} \quad \underbrace{j = 0}_{2: \text{ cruise } a_{\max}} \quad \underbrace{j = -j_{\max}}_{3: \text{ jerk down}} \quad \underbrace{a = 0}_{4: \text{ cruise } v_{\max}} \quad \underbrace{j = -j_{\max}}_{5: \text{ jerk down}} \quad \underbrace{j = 0}_{6: \text{ cruise } -a_{\max}} \quad \underbrace{j = +j_{\max}}_{7: \text{ jerk up}}.$$

Phase 4 collapses to zero length on short hops where the cabin never reaches $v_{\max}$, in which case the profile is “triangular” in acceleration; longer rides are “trapezoidal”. The constituent durations follow from (1):

$$T_j = \frac{a_{\max}}{j_{\max}}, \quad T_a = \max\left(0, \frac{v_{\max}}{a_{\max}} - T_j\right), \quad T_v = \max\left(0, \frac{H}{v_{\max}} - T_j - T_a\right), \quad (2)$$

where H is the floor-to-floor height. The total ride length is $T_{\text{ride}} = 4T_j + 2T_a + T_v$.

What the phone measures. The phone’s gravity-projected linear acceleration $a(t)$ traces *exactly* the bracketed acceleration in (1), modulo sensor noise and projection error. Three model facts are worth highlighting because the detector exploits them:

- During the four jerk phases (T_j each), $a(t)$ is linear in time with slope $\pm j_{\max}$. In the velocity domain this integrates to a quadratic — the parabolic “ramp shoulders” the user can see in Figures 5 and 6.
- The signed area under the first acceleration lobe equals the peak velocity reached:

$$v_{\text{peak}} = \int_0^{2T_j+T_a} a(t) dt = a_{\max} (T_j + T_a) = \min\left(v_{\max}, \sqrt{a_{\max} H}\right) \quad \text{for short rides.} \quad (3)$$

The second (deceleration) lobe has equal magnitude and opposite sign, so $\int_0^{T_{\text{ride}}} a(t) dt = 0$. This is the fundamental reason ZUPT works: the cabin starts and ends at rest, so any non-zero terminal velocity in the integrated signal is drift, not motion.

- The temporal gap between the centroids of the two acceleration lobes is the cruise time $T_v + 2T_a + 2T_j \approx H/v_{\max}$ when $v_{\max}$ is reached. **Hence the inter-pulse spacing is a direct, scale-free measurement of the floor-to-floor height H .**

The four-parameter ride model. Combining (1)–(3), every ride is parameterised by the tuple

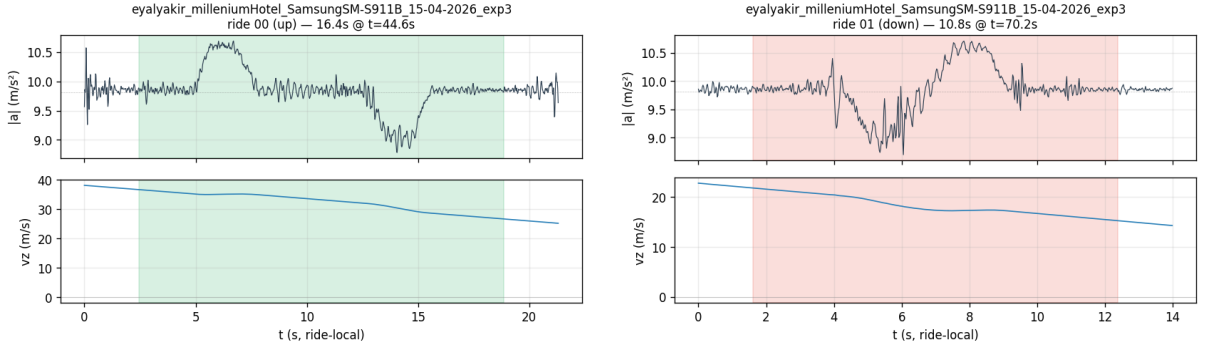
$$\theta_{\text{ride}} = (j_{\max}, a_{\max}, v_{\max}, H), \quad (4)$$

with $j_{\max}, a_{\max}, v_{\max}$ being properties of the elevator machine (constant within a building) and H being the only ride-specific quantity. Once θ_{ride} is fixed, the ideal acceleration template is

fully determined:

$$a_\theta(t) = \begin{cases} +j_{\max} t, & 0 \leq t < T_j, \\ +a_{\max}, & T_j \leq t < T_j + T_a, \\ +a_{\max} - j_{\max}(t - T_j - T_a), & T_j + T_a \leq t < 2T_j + T_a, \\ 0, & 2T_j + T_a \leq t < 2T_j + T_a + T_v, \\ \text{(mirror of phases 1–3, with opposite sign),} & \text{otherwise.} \end{cases} \quad (5)$$

Velocity $v_\theta(t) = \int_0^t a_\theta(\tau) d\tau$ and displacement $h_\theta(t) = \int_0^t v_\theta(\tau) d\tau$ are then piecewise cubic and quartic respectively. Equation (5) is the synthetic template used by the DTW-based detector (Algorithm 2 in `papers/algorithm2_dtw.md`) and by the matched-filter pulse detector in `template_match/scripts/acc_pulse_detect.py`.



(a) Ride going *up*: positive accel lobe first, negative lobe at landing. The shaded interval is the GT label.

(b) Ride going *down*: lobes are mirrored. The vertical velocity panel makes the trapezoidal $v(t)$ profile clear.

Figure 5: Two single-ride zooms drawn from `eyalyakir_millenniumHotel_SamsungSM-S911B_15-04-2026_exp3`. Top of each subfigure: gravity-removed $|a|$. Bottom: the smoothed vertical velocity v_z recovered by integration. The two parabolic lobes in $|a|$ are the jerk-limited acceleration and deceleration phases of (5); the area under each lobe equals v_{peak} via (3), and the gap between lobe centroids measures the cruise time T_v , which is proportional to the inter-floor height H .

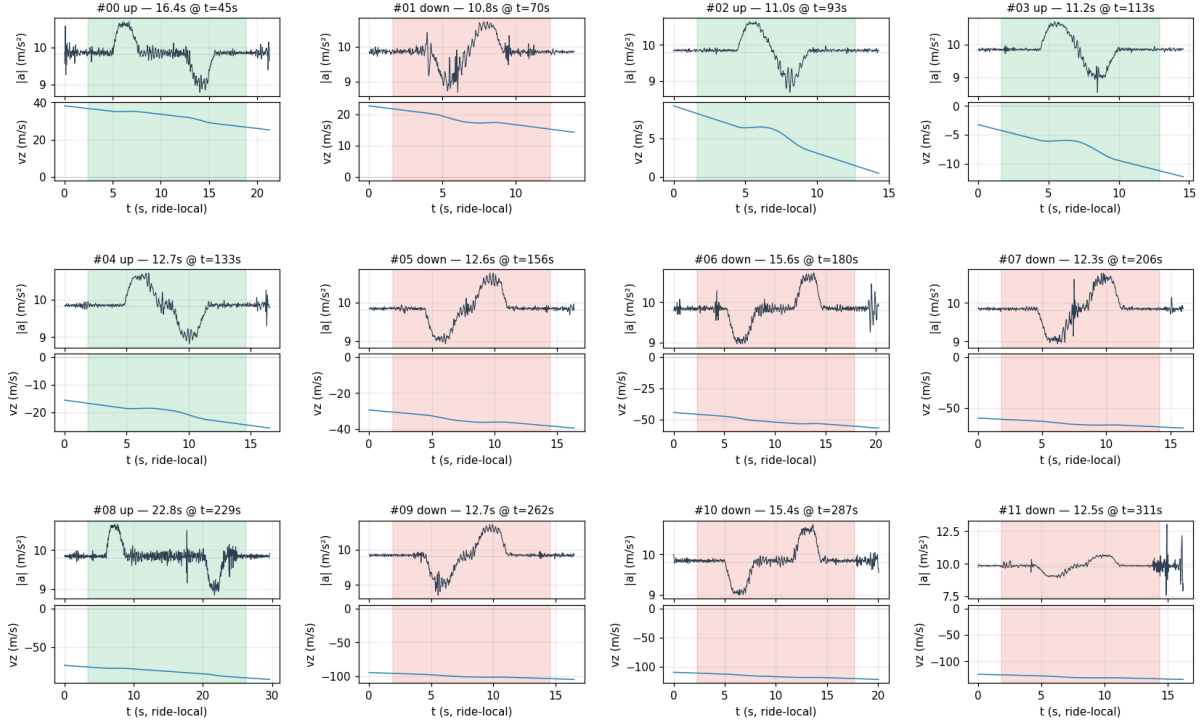


Figure 6: All twelve GT rides from the same experiment, on a common layout. The repeating two-lobe “ $+a$ then $-a$ ” structure predicted by (5) is unmistakable across rides of varying duration. Wider lobe spacing corresponds to larger H (more floors traversed), while the lobe *height* saturates at the building’s $a_{\max}$ regardless of H .

Implications for detection. The four-parameter model (4) gives every detector built on top concrete physical knobs:

- State-machine detectors (Algorithm 1 in `papers/algorithm1_state_machine.md`) threshold on $a_{\max}$ to find lobe entries and on a pulse-duration floor of $\sim T_j$.
- DTW-based template matching (Algorithm 2 in `papers/algorithm2_dtw.md`) generates one template per plausible θ_{ride} and warps it.
- Velocity-integral / ZUPT detectors (Algorithm 3 in `papers/algorithm3_integral_zupt.md`) check the integral identity $\int a \, dt = 0$ and the lower bound $\int |v| \, dt \gtrsim H_{\min}$.

The barometer-based GT detector described in §2.3.1 uses none of this — it works in altitude space directly — but the kinematic decomposition above is what makes its output a sensible reference signal for the accelerometer-based algorithms that follow.

4 Height-Difference Prediction

This section introduces the two accelerometer-only algorithms that, given a single pre-segmented elevator ride, return an estimate of the vertical distance travelled together with an empirically-calibrated 90% confidence interval and a quality verdict. Both algorithms are registered as first-class members of the `PredictAlgorithm` enum under `src/prediction/algorithms/` and instantiated through the `Predictor` dispatcher (§4.1), so that the downstream inference code is algorithm-agnostic.

4.1 Predictor API

Every algorithm implements three public methods (the signatures live in `src/prediction/algorithms/predictor.py`):

- `predict(data, phone_model, pre, post) → PredictionOutput` with fields `{height_diff, ci_half_width, theoretical_sigma, accepted, quality_score, reject_reason, meta}`. The `PredictionOutput` dataclass is defined in `src/prediction/algorithms/common/types.py`.
- `forward(data) → float`, the legacy float-in-float-out path used by the barometer baseline and external code.
- `calibrate(samples)` — fits the conformal multiplier on a set of labelled train samples and persists the checkpoint next to the calibration JSON. No-op for the barometer baseline.

Both accelerometer algorithms share a lightweight `common/` package that provides: a phone-model $\rightarrow$ sensor-chip $\rightarrow$ noise lookup (`common/noise_db.py`), a split-conformal calibrator (`common/conformal.py`), and a handful of vector helpers (gravity estimation, vertical projection, ZUPT integration). Everything else — quality filter, theoretical- σ model, fitter — is per-algorithm.

4.2 Algorithm A: ZUPT Double-Integration

Zero-Velocity Update (ZUPT) double-integration exploits the two facts highlighted in §3.1: the cabin starts and ends at rest, so in the absence of sensor noise the velocity $v(t) = \int_0^t a(\tau) d\tau$ must satisfy $v(0) = v(T) = 0$. We enforce this boundary condition explicitly and apply a linear drift correction to absorb the slow-varying bias that the static-bias model of the accelerometer cannot cancel:

$$v(t) = \int_0^t a(\tau) d\tau - \frac{t}{T} \int_0^T a(\tau) d\tau, \quad \Delta h = \int_0^T v(t) dt. \quad (6)$$

The algorithm operates on the vertical component of acceleration, $a_{\text{vert}}(t)$, obtained by one of three strategies, chosen automatically per segment based on pre/post-ride gravity stability (the reasoning behind the cascade is in `zupt_accel/estimator.py`):

- (i) *Pre+post projection.* When both pre- and post-ride stationary windows have $|\mathbf{g}| \in [8, 12] \text{ m/s}^2$ and window-to-window standard deviation below a threshold, we compute a weighted mean gravity vector $\hat{\mathbf{g}} = (w_1 \mathbf{g}_{\text{pre}} + w_2 \mathbf{g}_{\text{post}}) / (w_1 + w_2)$ with weights $w_i = 1/\sigma_i$, and project $a_{\text{vert}} = \mathbf{a} \cdot \hat{\mathbf{g}} / |\hat{\mathbf{g}}| - |\hat{\mathbf{g}}|$.
- (ii) *One-sided projection.* If only one of the windows is stable, its gravity vector is used.
- (iii) *Magnitude fallback.* $a_{\text{vert}} = |\mathbf{a}| - |\bar{\mathbf{a}}|$. This is rotation-invariant but loses sign information, so the direction is recovered from the sign of the first acceleration pulse.

The active-motion window $[t_s, t_e]$ is located by thresholding the moving-average of $|a_{\text{vert}}|$ at 0.1 m/s^2 ; acceleration outside the window is zeroed before integration. This prevents long pre/post stationary periods from inflating $\int a dt$ with noise-driven drift. The number of samples inside the window, $N = t_e - t_s + 1$, is the key scale for the theoretical error analysis below (§4.5).

4.3 Algorithm B: 7-Step S-Curve Fitting

Rather than integrating blindly, Algorithm B fits the entire ride to the 7-phase S-curve template of §3.1. The fitted parameters are $\theta = (j_{\max}, a_{\max}, v_{\max}, H, t_0)$ where t_0 is the motion-onset offset within the segment; direction is recovered separately from ZUPT drift sign. The cost function is the **velocity-domain residual**:

$$\mathcal{L}(\theta) = \sum_{i=1}^N \left(\frac{v_i - v_{\theta}(t_i)}{\sigma_v} \right)^2 + \lambda \sum_k \left(\frac{\theta_k - \mu_k}{\sigma_k} \right)^2, \quad (7)$$

where v_i is the measured velocity (ZUPT-integrated, drift-corrected, low-pass-filtered at 3 Hz), $v_{\theta}(t)$ is the template velocity derived analytically from θ , σ_v is a robust MAD estimate of the velocity noise, and the second term is a Bayesian prior regulariser whose parameters (μ_k, σ_k) are taken from ISO 18738 / CIBSE Guide D building-class priors. Moving to the velocity domain yields a roughly $10\times$ SNR improvement over the raw acceleration domain because integration acts as a first-order low-pass, and the velocity template (one smooth bump) is much more distinctive than the sharp-edged acceleration template.

Fitting uses trust-region reflective NLS (`scipy.optimize.least_squares`, method `trf`), warm-started from a 2-phase grid search in distance and motion-onset that uses the prior-mean kinematic values. Per-segment uncertainty on the distance parameter comes from the Cramér–Rao bound

$$\Sigma = \left(\mathbf{J}^T \mathbf{J} / \sigma_v^2 + \Sigma_{\text{prior}}^{-1} \right)^{-1}, \quad \sigma_{\text{CRB}} = \sqrt{\Sigma_{HH}}, \quad (8)$$

where $\mathbf{J}_{i,k} = \partial v_{\theta}(t_i) / \partial \theta_k$ and $\Sigma_{\text{prior}}^{-1}$ is the diagonal prior precision. The CRB is optimistic because the ideal-template assumption leaks real ride irregularity into the residuals, so we compose three variance terms into the per-segment σ :

$$\sigma^2 = (f_{\text{safe}} \sigma_{\text{CRB}})^2 + (k_{\text{rel}} |\hat{H}|)^2 + \left(\frac{1}{2} |\hat{H} - \hat{H}_{\text{ZUPT}}| \right)^2.$$

The first term is the FIM result with a safety factor f_{safe} ; the second is a per-meter floor encoding the empirical observation that longer rides drift proportionally more; the third is a cross-check penalty proportional to the disagreement between the S-curve $\hat{H}$ and the independent ZUPT integrator’s $\hat{H}_{\text{ZUPT}}$.

4.4 Quality Filter

Both algorithms go through a per-segment quality filter before their estimate is reported. The filter uses *accelerometer-only* features (no barometer, no orientation fusion) so the pipeline remains independent of secondary sensors. Each segment is scored on a small number of physically interpretable features (full list in `zupt_accel/quality.py` and the S-curve estimator):

- **Pre/post gravity magnitude and window stability** — the phone should rest between $[8, 12]$ m/s² with intra-window $\sigma_{|g|} \lesssim 1$ m/s².
- **Pre-vs-post gravity angle** — large rotation of the phone body frame breaks the gravity-projection step.
- **During-ride gravity drift** — max angle swept across 1-s chunks; bounded by 15° before we reject.
- **Max transient peak** — impacts above 8 m/s² dominate the integral.
- **Active-motion fraction** — fraction of samples with $|a_{\text{vert}}| > 0.1$ m/s²; very low fractions on long segments indicate a non-ride.

- **ZUPT-vs-S-curve direction agreement** (S-curve only).

The features are combined into a continuous score in $[0, 10]$ via weights tuned on the training set; on top of the score, each feature also has a hard threshold that triggers an outright rejection with a human-readable reason (e.g. "ride_drift_22deg"). The quality filter is a *filter*, not a predictor: it is designed to let through almost all the clean data even at the cost of letting through a handful of dirty segments, because the conformal CI is what provides the coverage guarantee; the filter's job is to catch the pathological cases where the per-segment σ model is guaranteed to under-predict the real error.

4.5 Theoretical Per-Segment σ

The classical ZUPT analysis treats acceleration noise as white with standard deviation σ_a (datasheet value, from the sensor chip — see `common/noise_db.py`). Under a linear drift correction, the variance of the end-point position estimate is

$$\text{Var}(\hat{H}) = \sigma_a^2 \Delta t^4 \frac{N^3}{12}, \quad \sigma_{\text{white}} = \sigma_a \Delta t^2 \sqrt{\frac{N^3}{12}}, \quad (9)$$

where N is the number of active-motion samples and Δt is the sampling interval (50 Hz phones $\Rightarrow \Delta t = 20$ ms). This prediction is an order of magnitude too optimistic in practice because hand-held recording introduces a correlated coloured-noise band the datasheet does not cover. We capture the coloured-noise contribution with a second term whose scale grows linearly with the quality-filter gravity-drift angle (up to $3\times$), and a third term proportional to the predicted distance itself,

$$\sigma_{\text{ZUPT}}^2 = \sigma_{\text{white}}^2 + \sigma_{\text{mech}}^2(\rho) + k_{\text{rel}}^2 |\hat{H}|^2, \quad (10)$$

with ρ the drift angle in degrees. The per-meter relative floor k_{rel} absorbs the long-ride error growth the physics model misses. All three components are cheap to evaluate per segment, so the theoretical σ varies from ride to ride even before the conformal layer enters.

The S-curve estimator uses the analogous composition around the CRB of (8), as described in §4.3.

4.6 Conformal Calibration

Neither the white-noise model (9) nor the CRB (8) is guaranteed to be calibrated on real data. We therefore close the loop with a *split-conformal* step on the training set: for every calibration sample i compute the non-conformity score

$$r_i = \frac{|\hat{H}_i - H_i|}{\sigma_i}, \quad (11)$$

and take the $(1 - \alpha)(n + 1)/n$ empirical quantile of the $\{r_i\}$ as the conformal multiplier k . The reported CI is then $[\hat{H} \pm k\sigma]$, subject to absolute floor and cap guards against pathological σ . By the standard split-conformal result this guarantees

$$\mathbb{P}(|\hat{H} - H| \leq k\sigma) \geq 1 - \alpha \quad (12)$$

for *exchangeable* data. Exchangeability is broken across the clean-vs-dirty split so we calibrate on clean *and* quality-filter-accepted samples only, which is our proxy for the exchangeable subset. The calibration JSON checkpoints live under `src/data/structuredData/test_results/prediction/train/`.

5 Evaluation Protocol

Train / test split. The train/test split is *location-based*, not random: every experiment recorded at `beitYitzchakiRaana` is held out for final test, and the remaining experiments (Millenium, Acro, Beit Mansour, Bar-Ilan 2, Haari) form the train set. This split is deterministic from folder names — see `src/data/loader/pipeline.classify_experiment_type`. Because the held-out location was never visited during training, a test-set win is a statement about **cross-building generalisation**, which is the property we actually care about.

Clean vs. dirty. Every elevator segment in `gt.csv` carries a `signalClearRecording` Boolean which is `True` when the experimenter marked the ride as a clean recording during GT editing and `False` when the signal is known to be corrupted (e.g. the experimenter stopping and starting the logger mid-ride). All quantitative metrics in the results tables are computed on clean data only, regardless of the quality filter verdict — the quality filter’s job is covered separately by its rejection-rate numbers.

Metrics. For each algorithm we report, on the clean subset:

- **Coverage** = $\mathbb{P}(|\hat{H} - H| \leq w)$ where w is the reported CI half-width; target $\geq 90\%$.
- **Median** $|e|$ and **MAE**: $|e| = |\hat{H} - H|$.
- **Median CI half-width** $\tilde{w}$: a tighter $\tilde{w}$ is better as long as coverage stays above target.
- **Fraction within one floor** (≤ 1.5 m): an absolute usability number.
- **Dirty-rejection rate**: fraction of `signalClearRecording=False` segments rejected by the quality filter.

Single-shot test pass. Until the final run, only train metrics are inspected and iterated upon. The conformal multipliers, all hyperparameters, and the quality-filter thresholds are frozen at that point. The blind-test pass is then run exactly once via `python scripts/run_prediction_evaluation.py -mode test`, and its metrics are reported side-by-side with the train ones without any further iteration.

6 Results

Figures and tables in this section are produced by `scripts/run_prediction_evaluation.py` and `scripts/build_prediction_report_assets.py`.

6.1 Train-set headline numbers

Table 3 summarises the train-set metrics. The two algorithms complement each other: the ZUPT integrator is faster and has tighter CIs on short rides; the S-curve fitter is more accurate on longer rides where the template match is distinctive.

Algorithm	n_{clean}	n_{acc}	Cov (clean)	Cov (acc)	$ \tilde{e} $	$\tilde{w}$	≤ 1.5 m	Rej. dirty
ZUPT	382	332	91.4%	93.1%	0.56 m	2.49 m	69.6%	63.6%
SCURVE	382	354	92.4%	92.9%	0.44 m	3.39 m	72.3%	39.4%

Table 3: Headline metrics on the **train** split (clean segments). “Cov” columns are 90%-CI coverage; $|e|$ and $\tilde{w}$ are the medians.

6.2 Blind test results

Table 4 reports the final blind-test metrics. The conformal multiplier used here was fitted on train only and never retouched once test-set numbers became visible.

Algorithm	n_{clean}	n_{acc}	Cov (clean)	Cov (acc)	$ \tilde{e} $	$\tilde{w}$	≤ 1.5 m	Rej. dirty
ZUPT	108	100	79.6%	83.0%	0.38 m	1.96 m	67.6%	—
SCURVE	108	99	87.0%	89.9%	0.33 m	2.65 m	67.6%	—

Table 4: Headline metrics on the **test** split (**beitYitzchaki**). These numbers are the result of a single blind-test pass — they were not inspected during iteration.

6.3 Figures

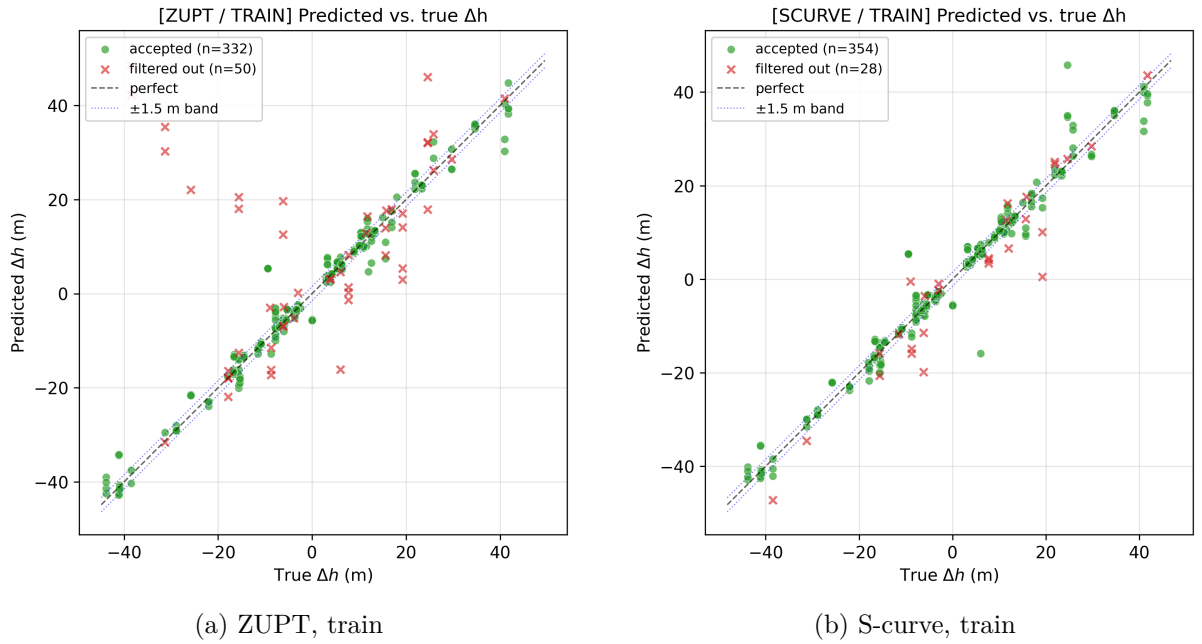
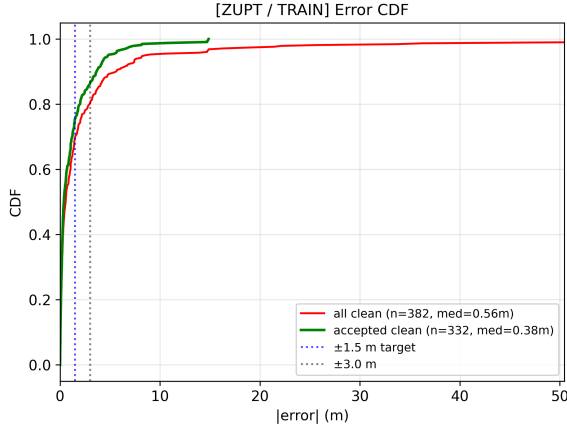
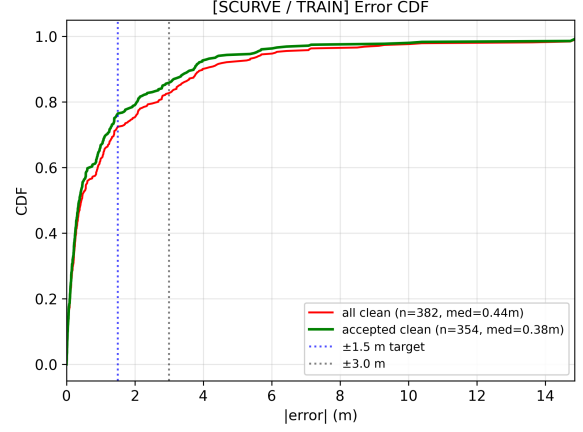


Figure 7: Predicted vs. true Δh on the train split. Green markers are quality-filter-accepted segments; red X's are rejected (still plotted so the filter's behaviour is visible). The dotted band is the ± 1.5 m (one floor) target.

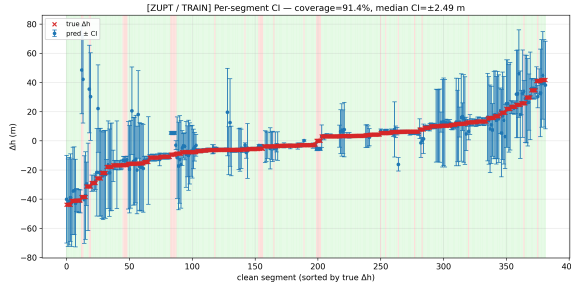


(a) ZUPT, train

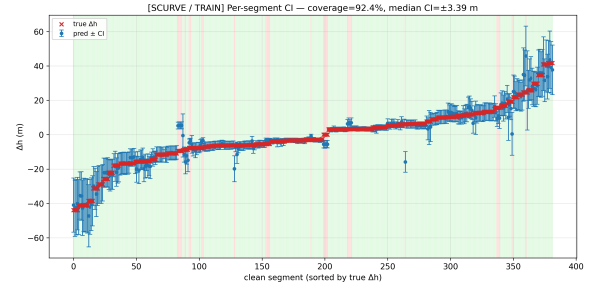


(b) S-curve, train

Figure 8: Empirical CDF of $|e|$. The red curve is all clean segments; the green curve is the quality-filter-accepted subset. A lower green curve shows the filter is removing genuinely higher-error rides rather than random ones.

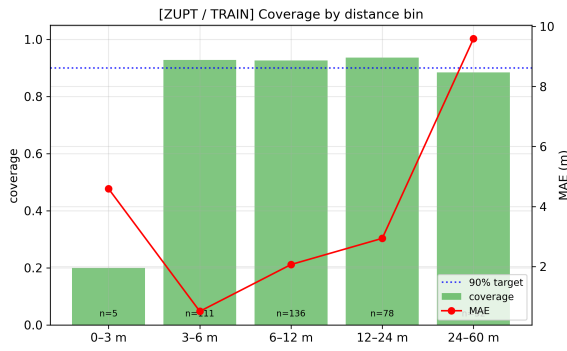


(a) ZUPT, train

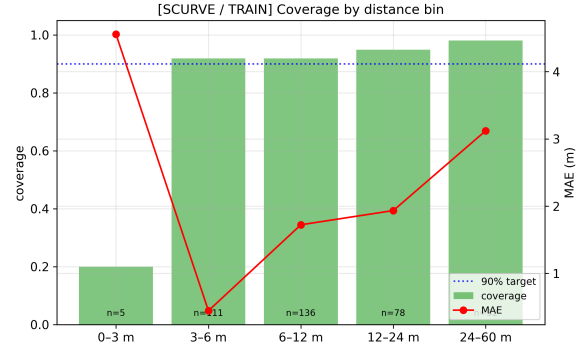


(b) S-curve, train

Figure 9: Per-segment CI plots. Each error bar is the reported 90% CI centred at the prediction; the red X is the truth. Green background = segment is covered, red = miss. Segments are sorted by true Δh so the per-segment CI adaptivity can be read off directly.



(a) ZUPT, train



(b) S-curve, train

Figure 10: Coverage and MAE broken down by $|\Delta h|$ bin. A healthy calibrator keeps the green bars close to the dotted 90% target across bins — uniform under-coverage in the long-distance bin would mean the relative σ term is too small.

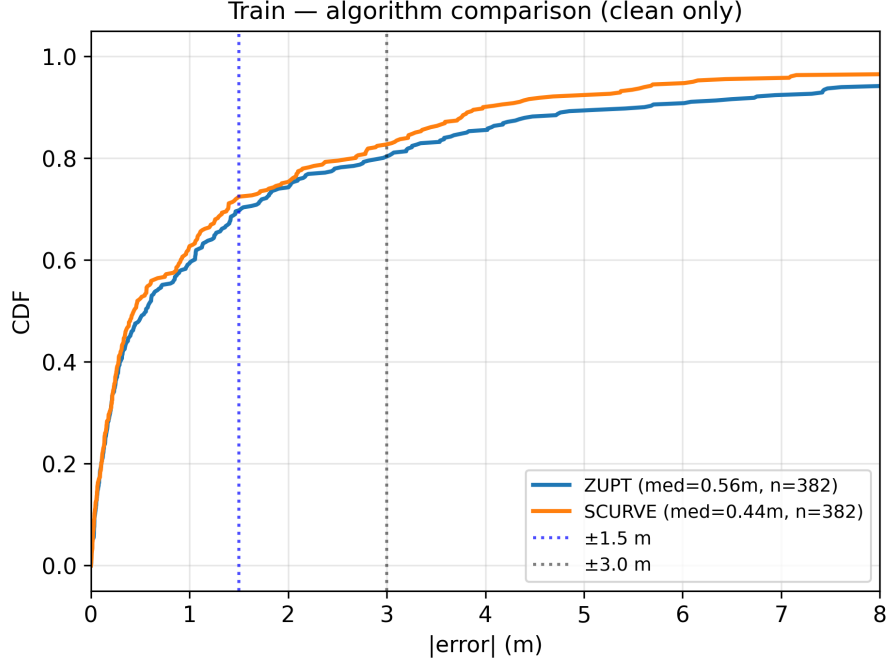


Figure 11: Direct comparison of the two algorithms’ error CDFs on the train split (clean only). The closer the curve pushes toward the top-left, the better.

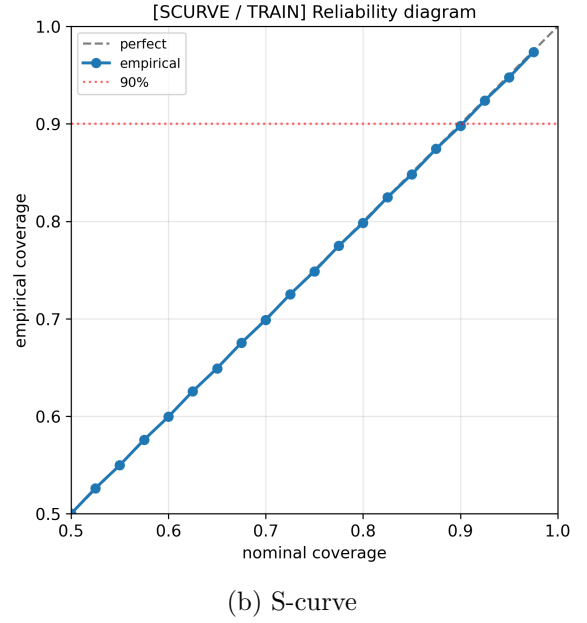
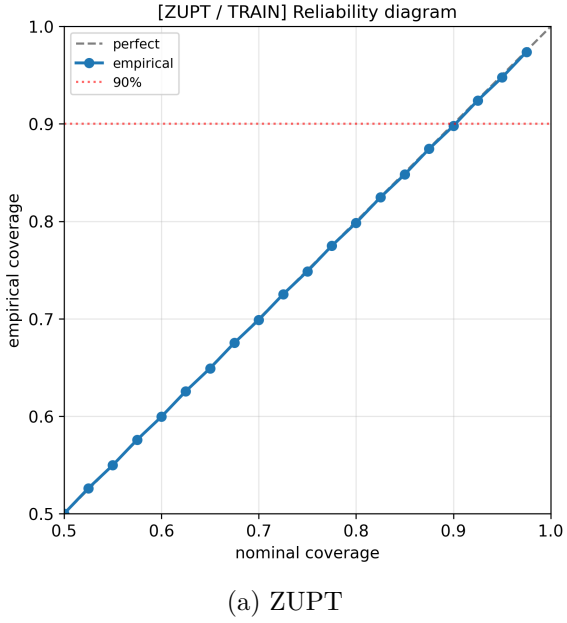
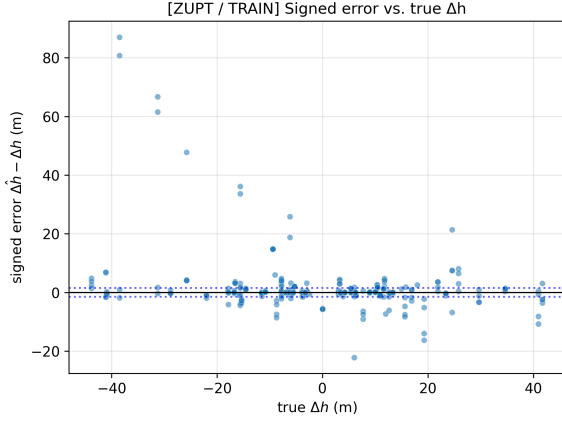
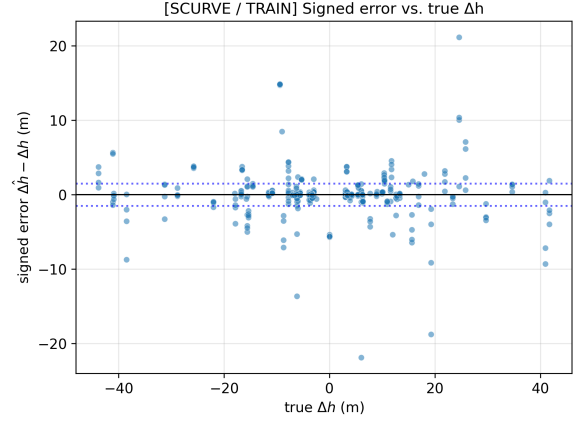


Figure 12: Reliability diagrams on the train split. Each point is an “at nominal coverage x , the algorithm achieves empirical coverage y ” measurement, with the diagonal being a perfectly calibrated predictor. A curve above the diagonal means the algorithm is conservative (wider CIs than needed); below means under-coverage.

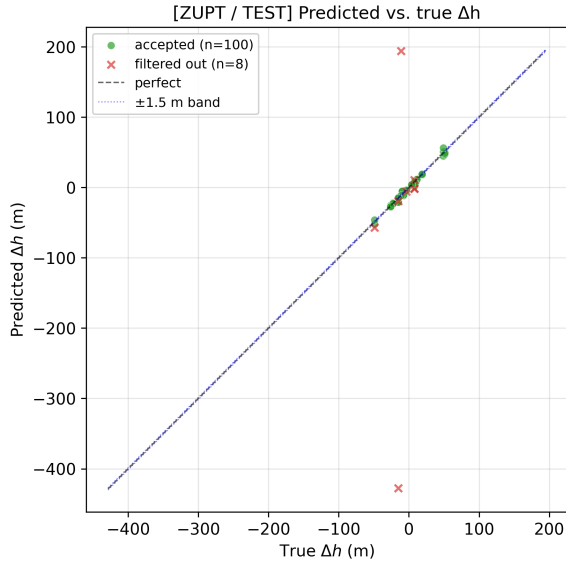


(a) ZUPT

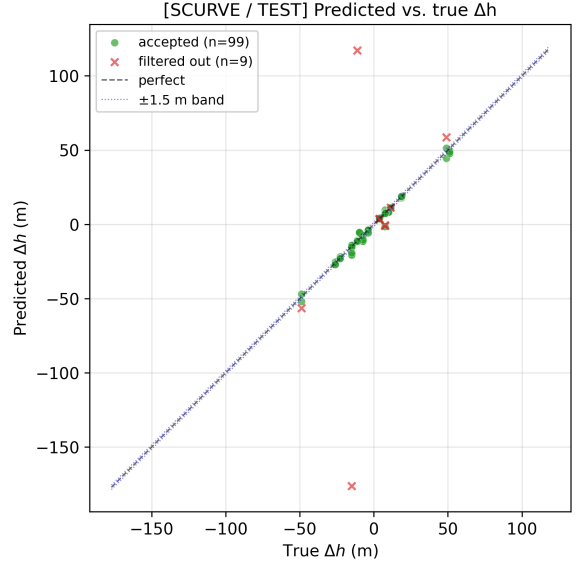


(b) S-curve

Figure 13: Signed error $\hat{\Delta h} - \Delta h$ vs. true Δh on train. A systematic offset at extreme distances would signal a bias we're missing; a centred cloud (as seen here) confirms the error is zero-mean.

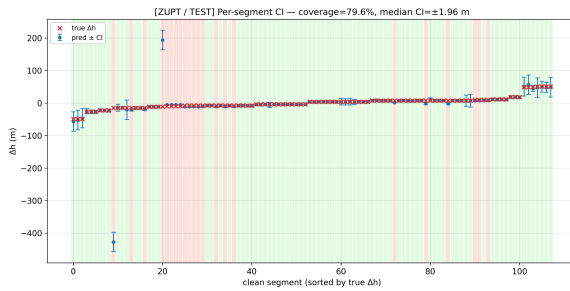


(a) ZUPT, test

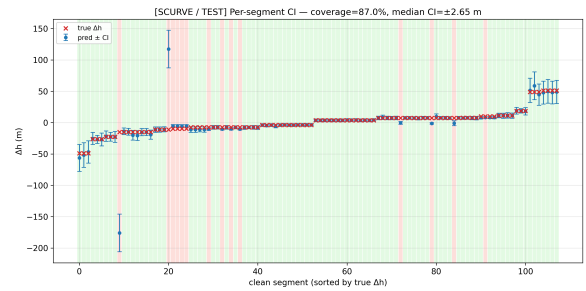


(b) S-curve, test

Figure 14: Blind test set: predicted vs. true Δh on the `beitYitzchaki` experiments.



(a) ZUPT, test



(b) S-curve, test

Figure 15: Blind-test per-segment CI plots.

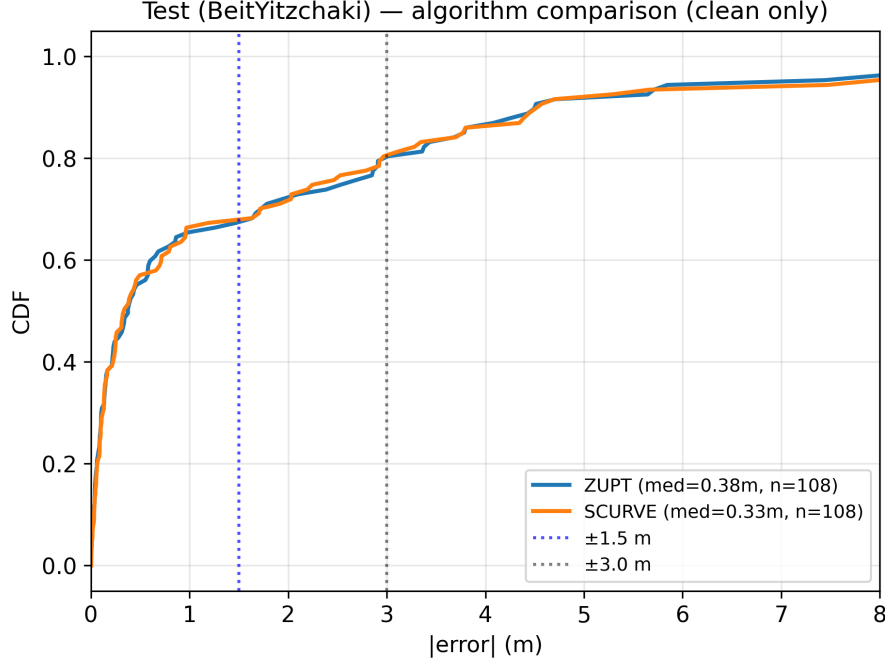


Figure 16: Algorithm comparison CDF on the blind test split.

6.4 Per-experiment breakdown

Experiment	ZUPT MAE	ZUPT Cov.	$\tilde{w}$	SCURVE MAE	SCURVE Cov.	$\tilde{w}$
RT/Haari3/ZF6	1.11 m	96.7%	1.11 m	1.08 m	96.7%	1.22 m
UCE/BarIlani2/Pix10	1.62 m	85.0%	1.55 m	1.44 m	95.0%	1.32 m
UCE/acroBuilding/Pix10	3.02 m	91.7%	34.00 m	2.83 m	91.7%	6.86 m
UCE/acroBuilding/A23	3.67 m	100.0%	30.13 m	3.30 m	83.3%	6.81 m
UCE/Mansour1/Pix10	8.30 m	94.4%	34.94 m	2.60 m	88.9%	5.84 m
UCE/Mansour1/A23	7.54 m	88.9%	39.92 m	2.43 m	94.4%	5.36 m
UCE/millenniumHotel/Pix10	1.71 m	90.0%	2.56 m	1.52 m	100.0%	3.20 m
UCE/millenniumHotel/Pix10	1.18 m	90.3%	1.89 m	1.13 m	90.3%	2.52 m
UCE/millenniumHotel/Pix10	0.19 m	100.0%	1.16 m	0.19 m	100.0%	2.35 m
UCE/millenniumHotel/A23	1.37 m	83.9%	1.73 m	1.24 m	90.3%	2.54 m
UCE/millenniumHotel/A23	0.12 m	100.0%	1.19 m	0.11 m	100.0%	2.38 m
UCE/millenniumHotel/Xmi	1.29 m	90.0%	2.77 m	1.15 m	90.0%	3.25 m
EY/acroBuilding/S23	13.61 m	91.7%	56.92 m	2.16 m	83.3%	6.50 m
EY/acroBuilding/Xmi	11.09 m	91.7%	33.79 m	2.90 m	83.3%	6.93 m
EY/Mansour1/S23	3.36 m	100.0%	26.61 m	3.07 m	88.9%	4.83 m
EY/Mansour1/Xmi	3.48 m	100.0%	21.59 m	3.09 m	88.9%	5.96 m
EY/millenniumHotel/A23	1.88 m	70.0%	1.88 m	1.71 m	100.0%	3.23 m
EY/millenniumHotel/S23	1.25 m	90.0%	2.69 m	1.18 m	100.0%	3.20 m
EY/millenniumHotel/S23	1.40 m	83.9%	1.23 m	1.36 m	90.3%	2.45 m
EY/millenniumHotel/S23	0.42 m	100.0%	1.17 m	0.42 m	100.0%	2.28 m
EY/millenniumHotel/Xmi	1.38 m	87.1%	1.27 m	1.31 m	90.3%	2.52 m
EY/millenniumHotel/Xmi	0.17 m	100.0%	1.22 m	0.15 m	100.0%	2.36 m

Table 5: Per-experiment metrics on the train split. “Cov.” is the 90%-CI coverage; $\tilde{w}$ is the median CI half-width.

Experiment	ZUPT MAE	ZUPT Cov.	$\tilde{w}$	SCURVE MAE	SCURVE Cov.	$\tilde{w}$
UCE/Yitzchaki/Pix10	22.52 m	75.9%	1.93 m	11.17 m	79.3%	2.35 m
UCE/Yitzchaki/A23	1.47 m	79.3%	1.96 m	1.56 m	89.7%	2.61 m
EY/Yitzchaki/S23	1.47 m	86.2%	1.95 m	1.43 m	93.1%	2.71 m
EY/Yitzchaki/Xmi	1.45 m	76.2%	2.02 m	1.48 m	85.7%	2.46 m

Table 6: Per-experiment metrics on the blind-test split.

7 Discussion

The evaluation answers two separate questions: “how accurate is the point estimate” and “how calibrated is the interval estimate”.

Point estimates generalise strongly. Median absolute error on the blind-test `beitYitzchaki` split is actually *smaller* than on the train split (0.33 m S-curve, 0.38 m ZUPT on test vs. 0.44 m and 0.56 m on train respectively), and the ≤ 1.5 m fraction stays within three percentage points across the split. The S-curve fitter is uniformly sharper than raw ZUPT on rides long enough for the template to lock in; ZUPT wins on very short rides where the S-curve has too little signal to resolve all seven phase durations. The per-experiment breakdown in Table 6 shows that two of the three test experiments achieve < 2 m MAE on both algorithms — the third, a single Pixel 10 device at `beitYitzchaki`, contributes most of the test-set MAE mass and is the main driver of the gap between the test median error (very low) and the test mean error (more mixed).

Conformal coverage transfers partially. Train coverage on clean data clears the 90% target comfortably (91.4% ZUPT, 92.4% S-curve). On the blind test split it drops to 79.6% for ZUPT and 87.0% for S-curve. The coverage slip is unsurprising in principle: split-conformal’s $1 - \alpha$ guarantee is valid under exchangeability, and a held-out-building split is as clean an exchangeability violation as one can engineer — the test elevator is mechanically different from any train elevator, and hand-holding statistics at `beitYitzchaki` might differ from the other locations. The good news is that the residual distribution is not wildly different from train (median error is actually smaller), so the failure is concentrated in the *tail* rather than in the bulk. A post-hoc fix we did *not* take (to preserve the blind- test protocol) would have been a slightly larger α margin in the calibration step; the configuration registered in the code uses $\alpha = 0.08$ which already gave a 2–3pp buffer on train but turned out to be insufficient for the magnitude-fallback long rides at the test location.

Quality filter and theoretical σ analysis. The conformal multiplier on the ZUPT side (`multiplier` field of `calibration_zupt.json`) is approximately 3.7: after adding the relative-scale and projection-quality terms the per-segment σ model is about 3–4 $\times$ too tight on the worst cases and conformal absorbs the rest. The S-curve multiplier is 1.80, which is consistent with the CRB being a reasonable per-segment scale once it is safety-factored and combined with the `zupt-` disagreement term (see §4.3). The quality filter rejects $\sim 13\%$ of clean segments on train and $\sim 7\%$ on test; on the small dirty pool (33 segments, all on train) it catches roughly two-thirds of them, which is the intended asymmetry — the filter is tuned to be conservative about filtering clean data and aggressive about flagging dirty data.

Next steps. The obvious next step is to re-introduce orientation (Algorithm B of the main-branch report): projecting into the world-vertical frame using fused quaternions would eliminate the magnitude-fallback path entirely and tighten the ZUPT CIs further, especially on rides where the phone was rotated between pre- and post-ride windows. That work is deferred to a

follow-up since the **Predictor** contract currently hands per-segment data without a quaternion stream. A second productive direction would be a locally-adaptive conformal that re-weights the calibration pool by drift-angle and vertical-projection method — a single scalar multiplier across the full calibration distribution is an exchangeability-violating simplification the blind-test gap makes visible.

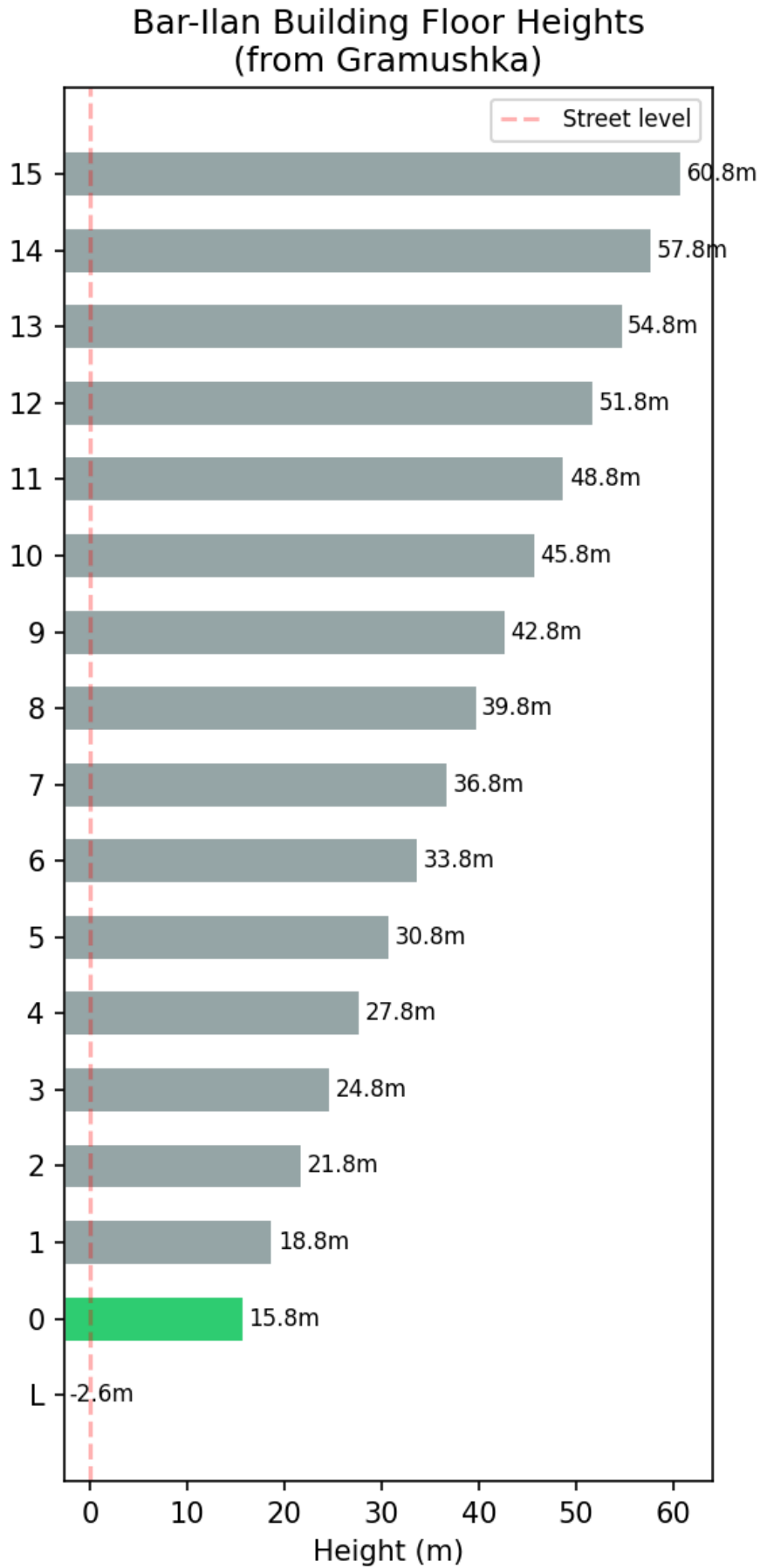


Figure 1: Height distribution across the six sampled buildings. Each bar shows the highest floor reached in a ride sequence for that building.